Reviewer Pack — Minimal Quadratic Residue Representation and Communication C…

Entry e47120e78748 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 02:23:09 UTC

Minimal Quadratic Residue Representation and Communication Complexity Growth for k-Colorable Graphs

Entry ID: `e47120e78748` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 02:23:09 UTC

1. Conjecture

Field A (mathematical branch): Quadratic Residue Theory **Field B** (complexity object): Communication Complexity

Statement:

> For every k-colorable graph G with n vertices, the minimal quadratic residue representation (mqr(G)) of G is linearly correlated with its communication complexity growth rate for distinguishing k-colorings, such that mqr(G) = Θ(growth_rate(G)).

Rationale (proposer's reasoning):

> Quadratic Residue Theory could provide a unique way to encode combinatorial structures like graphs, potentially offering insights into the complexity of tasks like communication. Since communication complexity measures the minimum number of bits required for two parties to agree on information, it is an important complexity measure. The conjecture suggests that certain properties in quadratic residue theory might be related to communication complexity growth rate for specific graph problems, which could expose new connections between these areas.

Taxonomy category: `Quadratic_Residue_Theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8aad1363563c6b2e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the Pearson correlation coefficient between mqr(G) and growth_rate(G) for all k-colorable graphs G with n vertices (n ≤ 40) is statistically significant at a p-value threshold of < 0.05, computed over 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def is_k_colorable(graph, k):
    n = len(graph)
    colors = [-1] * n

    def dfs(node, color):
        if node in visited:
            return True
        visited.add(node)
        colors[node] = color

        for neighbor in graph[node]:
            if colors[neighbor] == color or not dfs(neighbor, (color + 1) % k):
                return False
        return True

    visited = set()
    for i in range(n):
        if node not in visited:
            if not dfs(i, 0):
                return False
    return True

def generate_k_colorable_graph(n, k):
    graph = [[] for _ in range(n)]
    colors = [random.randint(0, k-1) for _ in range(n)]

    def add_edge(u, v):
        if u != v and (v not in graph[u] or u not in graph[v]):
            graph[u].append(v)
```

```
            graph[v].append(u)

    while True:
        for i in range(n):
            for j in range(i+1, n):
                if colors[i] == colors[j]:
                    add_edge(i, j)
        if is_k_colorable(graph, k):
            break
        graph = [[] for _ in range(n)]

    return graph

def minimal_quadratic_residue_representation(graph):
    n = len(graph)
    mqr = 0

    def quadratic_residue(x):
        return x * x % (n + 1)

    for i in range(n):
        for j in range(i+1, n):
            if j in graph[i]:
                mqr += quadratic_residue(j - i)

    return mqr

def communication_complexity_growth_rate(graph, k):
    n = len(graph)
    growth_rate = 0

    def dfs(node, visited):
        nonlocal growth_rate
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                growth_rate += 1
                dfs(neighbor, visited)

    for i in range(n):
        visited = set()
        dfs(i, visited)

    return growth_rate

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    k = random.randint(2, min(n, 6))
    graph = generate_k_colorable_graph(n, k)

    mqr = minimal_quadratic_residue_representation(graph)
    growth_rate = communication_complexity_growth_rate(graph, k)

    if growth_rate == 0:
        return {
```

```
            "metric_name": "correlation_coefficient",
            "metric_value": None,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "growth_rate_is_zero"
        }

    correlation_coefficient = (mqr * growth_rate) / (math.sqrt(mqr ** 2 + growth_rate ** 2))

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": abs(correlation_coefficient) >= 0.95,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='correlation_coefficient_not_significant' first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d1b31347.py", line 128, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d1b31347.py", line 96, in run_trial
    graph = generate_k_colorable_graph(n, k)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d1b31347.py", line 54, in generate_k_co
```

```
    if is_k_colorable(graph, k):
       ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d1b31347.py", line 35, in is_k_colorabl
    if node not in visited:
       ^^^^
NameError: name 'node' is not defined. Did you mean: 'None'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	24063
2	propose	ollama_remote	glm4:latest	0	0	24122
3	preregistration	ollama_remote	glm4:latest	0	0	23102
4	novelty	ollama_remote	glm4:latest	0	0	8333
5	novelty	ollama_remote	glm4:latest	0	0	17370
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18829
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18480
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11135
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19243
10	judge	ollama_remote	glm4:latest	0	0	12260

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 176938 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/e47120e78748.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e47120e78748.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e47120e78748.tar.gz` (if generated)